

Reflection

1. Requirements Elicitation with AI vs. Human Stakeholders

The iRE Agent consistently provided clear and structured descriptions, often listing key points, while human stakeholders were more unstructured and “thinking out loud.” The AI responses were concise and precise, reducing the risk of misinterpreting assumptions, whereas human responses sometimes required interpretation. While AI lacks human experiences like frustration or satisfaction, it offers practical advantages: elicitation is faster, text is automatically transcribed, and there is no fatigue. This allows unlimited follow-up questions without concern for the AI’s well-being, unlike with human stakeholders, where time and careful question selection limit questioning.

2. Using AI to Understand Web Development Concepts

AI made it easy to generate a working application from the requirements, which improved productivity but limited our understanding of the underlying architecture and frameworks, since the AI handled most of the implementation. It was also used to identify code smells during review, explaining why each was an issue and categorizing their severity. This clarified important concepts in code quality, though relying on AI may reduce one’s own ability to spot issues independently.

3. AI-Assisted Code Generation Process

GitHub Copilot was used as the primary tool for code generation. Most of the implementation was generated in a single pass using a well-structured Markdown instruction file derived from the project requirements. By translating requirements into clear and contextualized prompts, Copilot was able to produce most of the application with minimal refinement.

4. Revision and Correction of AI-Generated Code

The revision process differed from traditional debugging, as we used GitHub Copilot to implement fixes while we focused on identifying issues and defining changes. Most modifications were due to runtime errors and usability issues, such as missing environment configuration and incomplete handling of edge cases.

A key limitation was that Copilot did not inherently ensure good architectural quality. This became clear during code review, where many code smells were identified through both manual and AI inspection. This suggests that the initial generation prioritized functional correctness over maintainability, likely because code quality was not explicitly specified in the instruction file. As a result, the code worked as intended but required further refinement to meet standard engineering practices

5. Quality Assessment of AI-Generated Code

The AI-generated code worked correctly and met most requirements, with only minor bug fixes needed. It was well-structured, with repositories, services, and controllers clearly

organized. However, cohesion was sometimes low, mixing database access with business logic, and there was noticeable code duplication, showing that AI prioritized functionality over maintainability. The code was readable and functional, but improvements would be needed for long-term maintainability.

6. Best Practices Developed When Working with AI

One of the most effective practices was using well-structured instruction files as prompts. By translating requirements into clear and detailed documents, Copilot was able to generate most of the codebase in a single pass, reducing ambiguity and improving consistency.

Another important practice was treating AI as an implementation tool rather than a decision-maker. We identified issues such as runtime errors and code smells ourselves, then provided precise instructions for Copilot to implement fixes, especially during code review.

A key lesson was the importance of explicitly specifying non-functional requirements like maintainability and code quality. Since the initial prompts focused mainly on functionality, the generated code contained code smells, showing that AI performs best when all constraints are clearly defined.

7. Professional Identity and Way of Working

Working with AI shifted our role from primarily coding to analyzing, specifying, and validating solutions. Copilot handled most of the implementation, which increased productivity and confidence in producing runnable software, but it also highlighted that functional output does not guarantee quality. Responsibility remained somewhat high, especially in identifying runtime errors, code smells, and maintainability issues, showing that critical judgement is essential even when AI generates the code.

AI supported engineering thinking by explaining code smells and suggesting improvements, but it also risked encouraging superficial solutions, focusing on getting things to work rather than understanding the system deeply.

AI Interaction Log

19.01.26

group01@tdt4242.ntnu.no:

From the perspective of NTNU, what problems does the ai guide book want to address for the students regarding generative AI?

13:20:34

iRE agent:

The AI Guidebook aims to address several key issues for students at NTNU regarding generative AI. Firstly, it seeks to provide clear guidance on the responsible and ethical use of AI tools, ensuring compliance with academic integrity policies. Secondly, it addresses the need for students to effectively document and reflect on their AI usage to maintain transparency. Lastly, the guidebook aims to facilitate automated compliance checks to help students navigate institutional rules, thereby supporting their academic work while upholding ethical standards.

13:20:37

group01@tdt4242.ntnu.no:

Which of these problems are most problematic? why?

13:21:17

iRE agent:

The most problematic issues are the need for clear guidance on ethical AI use and the effective documentation of AI usage. Without clear guidance, students may unintentionally violate academic integrity policies, leading to serious consequences. Additionally, inadequate documentation can obscure the extent of AI assistance in their work, undermining transparency and accountability. Both issues are critical for fostering a responsible academic environment and ensuring that students maintain their learning integrity while using AI tools.

13:21:19

group01@tdt4242.ntnu.no:

From the perspective of an academic institution, what problems does the AI Guide book want to address for the students regarding generative AI?

13:22:06

iRE agent:

The AI Guidebook aims to address several key issues for students regarding generative AI. First, it seeks to enhance understanding and responsible usage of AI tools, ensuring students are aware of ethical considerations and academic integrity policies. Second, it provides a structured framework for students to log AI usage and declare it in assignments, promoting transparency. Lastly, the guidebook facilitates compliance checks against institutional rules, helping students navigate the complexities of AI support while maintaining their academic responsibility and integrity.

13:22:09

group01@tdt4242.ntnu.no:

Which of these problems are most problematic? why?

13:22:15

iRE agent:

The most problematic issue is the lack of understanding and responsible usage of AI tools. This concern is critical because students may inadvertently violate academic integrity policies, which can lead to serious consequences like plagiarism accusations. Additionally, without a clear framework for logging AI usage and declaring it, students might struggle with transparency, making it difficult for institutions to uphold ethical standards. Addressing these issues is vital to ensure that AI tools are integrated into academic practices without compromising educational values.

13:22:18

group01@tdt4242.ntnu.no:

What kind of data should the AI usage documentation consist of when a student registers this?

13:22:30

iRE agent:

The AI usage documentation should include the following data elements:

1. **Date and Time**: When the AI tool was used.
2. **Tool Name**: Identification of the specific AI tool used (e.g., ChatGPT, GitHub Copilot).
3. **Assignment Reference**: Linking usage to specific assignments or tasks.
4. **Purpose of Use**: A brief explanation of the AI tool's role in the task.
5. **Output Summary**: Key outputs or results generated by the AI.
6. **Ethical Considerations**: Any ethical concerns or considerations noted during usage.

This structured documentation will promote transparency and facilitate compliance with institutional policies.

13:22:34

group01@tdt4242.ntnu.no:

Should the student do this manually, or should it be automated?

13:22:48

iRE agent:

The process should be largely automated to enhance usability and reduce the burden on students. Automation can streamline data capture, ensuring accuracy and consistency in logging AI usage. However, students should still be prompted to provide context for their usage, such as ethical considerations and the purpose of AI application, as this fosters critical thinking and accountability. A hybrid approach balances ease of use with the need for personal reflection on AI tool application.

13:22:50

group01@tdt4242.ntnu.no:

How would a student today find out if their use of AI was allowed or not?

13:23:43

iRE agent:

A student can find out if their use of AI is allowed by consulting their institution's academic integrity policies, which typically outline acceptable practices regarding technology use. They can also seek guidance from faculty members or academic advisors who can provide insights into specific rules and expectations. Additionally, the AI Guidebook serves as a resource, offering ethical guidelines and compliance checks that clarify permissible AI usage in academic work.

13:23:46

group01@tdt4242.ntnu.no:

How does NTNU's policies currently regulate AI usage?

13:24:44

iRE agent:

I cannot provide specific details about NTNU's policies on AI usage. However, students are encouraged to review the official academic integrity guidelines published by NTNU, which typically address acceptable technology use, including generative AI. Engaging with faculty or academic advisors may also clarify how these policies are applied in practice.

13:24:45

group01@tdt4242.ntnu.no:

What AI policy from NTNU does students violate the most and why?

13:24:57

iRE agent:

I cannot specify which particular AI policy students at NTNU violate the most. Generally, violations may arise from misunderstandings regarding plagiarism and the proper attribution of AI-generated content. Students might not fully grasp the boundaries of acceptable usage, leading to unintended breaches of academic integrity. Encouraging thorough understanding of the guidelines and providing clear examples can help mitigate these issues.

13:25:00

group01@tdt4242.ntnu.no:

What kind of visualizations should the dashboard consist of?

13:25:11

iRE agent:

The dashboard should include the following visualizations:

1. **Usage Trends**: A time-based graph showing frequency and duration of AI tool usage over time.
2. **Tool Utilization**: Pie charts or bar graphs depicting the distribution of different AI tools used across assignments.
3. **Assignment Breakdown**: A visualization that links AI usage to specific assignments, highlighting which tasks had AI involvement.
4. **Compliance Status**: Visual indicators (e.g., green/red indicators) showing whether AI usage aligns with institutional policies.
5. **Ethical Reflection Summary**: A visual representation of ethical considerations noted by the student during AI usage, perhaps using tags or heat maps.

These visualizations will help students monitor their AI use and ensure adherence to academic integrity standards.

13:25:15

group01@tdt4242.ntnu.no:

When should the automated compliance checks run? During usage of the AI, or in certain intervals.

13:25:45

iRE agent:

Automated compliance checks should ideally run in real-time during AI usage to provide immediate feedback on adherence to academic integrity policies. This allows students to correct any potential issues as they arise. Additionally, periodic checks could be conducted at set intervals (e.g., after assignment submission) to ensure ongoing compliance and to review historical usage patterns. This dual approach balances proactive monitoring with retrospective analysis.

13:25:48

group01@tdt4242.ntnu.no:

What should happen if a violation is found?"

13:25:57

iRE agent:

If a violation is found, the system should initiate a notification process that alerts the student about the specific nature of the violation. This should be followed by a review mechanism where the student can provide context or justification for their AI usage. Depending on the severity of the violation, the institution may implement corrective actions, which could include educational interventions, discussions with academic advisors, or formal disciplinary measures in accordance with institutional policies. This approach promotes learning and accountability while upholding academic integrity.

13:25:59

group01@tdt4242.ntnu.no:

What ethical, legal or institutional constraints must the system respect when handling student data and/or AI usage logs?

13:26:05

iRE agent:

The system must respect several ethical, legal, and institutional constraints, including:

1. **Data Privacy Regulations**: Compliance with laws such as GDPR, which governs the handling of personal data, ensuring that student information is collected, processed, and stored securely.
2. **Institutional Policies**: Adherence to NTNU's academic integrity and data management policies, which dictate how student data should be handled and protected.
3. **Informed Consent**: Ensuring that students are adequately informed about how their data will be used and obtaining their consent before logging AI usage.

4. **Anonymization**: Where feasible, data should be anonymized to protect student identities, especially in analytics and reporting.

5. **Transparency**: Providing clear guidelines on how student data will be used and allowing students to access their data logs.

These constraints are essential for maintaining trust and compliance within the educational environment.

13:26:09

group01@tdt4242.ntnu.no:

How should privacy and explainability be balanced?

13:26:12

iRE agent:

Balancing privacy and explainability involves ensuring that users understand how their data is used while protecting their personal information. This can be achieved by:

1. **Data Minimization**: Collect only the essential data needed for functionality, reducing exposure of sensitive information.
2. **Clear Communication**: Provide transparent explanations regarding how data is utilized, what is logged, and the purpose behind it, without revealing personally identifiable information.
3. **User Control**: Allow students to manage their privacy settings, enabling them to choose what data they share and receive explanations about.
4. **Aggregated Insights**: Use aggregated data for analytics and compliance checks to maintain privacy while still providing insights into AI usage patterns.

This approach fosters trust while ensuring that students have access to the necessary information to understand the system's operations.

13:26:15

group01@tdt4242.ntnu.no:

How would you define a successful AI Guidebook in terms of learning outcomes, functionality and AI usage?

13:26:21

iRE agent:

A successful AI Guidebook can be defined by the following criteria:

1. **Learning Outcomes**: Students demonstrate a clear understanding of ethical AI usage, can articulate the implications of AI in their academic work, and show improved academic integrity through responsible tool application.
2. **Functionality**: The system provides intuitive logging, seamless integration of AI tools, real-time compliance checks, and accessible guidelines, ensuring that students can easily manage their AI usage while receiving timely feedback.
3. **AI Usage**: Students actively engage with the Guidebook, consistently log their AI interactions, and utilize the insights provided to reflect on their practices, leading to enhanced accountability and informed decision-making regarding AI tools.

These elements collectively ensure that the Guidebook not only supports academic success but also fosters a culture of responsible AI usage among students.

13:26:24

group01@tdt4242.ntnu.no:

Are there any measurable KPI's the system should track?

13:26:28

iRE agent:

Yes, the system should track the following measurable KPIs:

1. **User Engagement Rate**: Percentage of students actively using the Guidebook over a specified period, indicating its adoption and relevance.
2. **Compliance Rate**: Ratio of AI usage instances that meet institutional guidelines versus total usage, reflecting adherence to academic integrity policies.
3. **Reflection Submission Rate**: Percentage of students providing ethical reflections on their AI usage, which measures engagement with critical thinking and accountability.
4. **Usage Patterns**: Analysis of the frequency and type of AI tools used across different assignments, helping identify trends and areas needing further guidance.
5. **Feedback Response Time**: Average time taken to notify students about potential compliance issues, which impacts the effectiveness of real-time checks.

Tracking these KPIs will help assess the system's effectiveness in promoting responsible AI usage and guiding student learning.

13:26:34

group01@tdt4242.ntnu.no:

Whats the functional requirments 1:37:43 PM?

13:37:56

iRE agent:

To identify functional requirements, we need to focus on the specific capabilities the system must provide. Based on the project context, here are some functional requirements:

1. The system shall allow students to log their AI tool usage.
2. The system shall enable students to declare AI usage for each assignment.
3. The system shall provide ethical guidelines for AI usage.
4. The system shall conduct automated compliance checks against institutional rules.
5. The system shall feature a personal dashboard to visualize AI usage patterns over time.

If you have specific areas you want to explore further or additional requirements to consider, let me know.

13:37:59

21.01.26

group01@tdt4242.ntnu.no:

From the perspective of an academic institution, what problems does the AI Guidebook aim to address for students regarding the use of generative AI?

09:00:18

iRE agent:

The AI Guidebook aims to address several key problems for students within an academic institution. First, it provides clarity on how to use generative AI tools responsibly, ensuring that students adhere to academic integrity policies. Second, it helps mitigate the risk of unintentional plagiarism or misrepresentation by establishing guidelines for AI usage. Third, it fosters an understanding of ethical considerations surrounding AI, promoting responsible engagement with technology. Lastly, the application supports transparency and traceability in AI usage, enabling institutions to monitor compliance with their academic standards.

09:00:21

group01@tdt4242.ntnu.no:

Which of these problems are currently the most critical, and why do they require institutional intervention?

09:00:25

iRE agent:

The most critical problems currently are the risk of unintentional plagiarism and the need for clarity on responsible AI use. Institutional intervention is essential because these issues directly impact academic integrity, which is foundational to educational credibility. Without clear guidelines and support, students may inadvertently violate policies, leading to severe academic consequences. Additionally, fostering ethical considerations requires a structured approach from institutions to ensure students understand the implications of their AI usage in the academic context.

09:00:28

group01@tdt4242.ntnu.no:

How do these challenges affect academic integrity, learning outcomes, and trust between students and the institution?

09:00:32

iRE agent:

These challenges undermine academic integrity by increasing the likelihood of plagiarism and misrepresentation, which can lead to disciplinary actions that affect students' academic records. Learning outcomes may suffer as students might rely too heavily on AI tools instead of developing critical thinking and problem-solving skills. This reliance can create a disconnect between students and the institution, eroding trust if students feel unsupported or uncertain about the acceptable use of technology. Institutions must address these issues to maintain credibility, support ethical learning environments, and foster a positive relationship with their students.

09:00:34

group01@tdt4242.ntnu.no:

How do students today typically find out whether the use of AI is allowed or prohibited in a specific course or assignment?

09:00:41

iRE agent:

Students typically discover the policies regarding AI usage through course syllabi, institutional guidelines, or direct communication with instructors. However, the information may not always be clearly outlined or accessible, leading to confusion. Often, students may also rely on informal channels such as peer discussions or online forums, which can result in inconsistent understanding of what is permissible. This highlights the need for a centralized, clear resource, such as the AI Guidebook, to ensure all students have accurate and consistent information regarding AI use in their academic work.

09:00:46

group01@tdt4242.ntnu.no:

What shortcomings do you see in the way current institutional policies regulate or communicate AI usage to students?

09:00:49

iRE agent:

Current institutional policies often lack clarity and specificity regarding AI usage, leading to vague interpretations. Additionally, these policies may not be regularly updated to reflect the rapid evolution of AI technology, resulting in outdated guidelines. Communication channels can be inconsistent, with information sometimes buried in lengthy documents or not effectively disseminated to students. This lack of a centralized, easily accessible resource hinders students' ability to understand and comply with expectations, making it critical to establish a structured approach like the AI Guidebook to address these shortcomings.

09:00:52

group01@tdt4242.ntnu.no:

What kind of information should students be required to document regarding their AI usage when submitting assignments?

09:00:56

iRE agent:

Students should be required to document the specific AI tools used, the nature of their assistance (e.g., drafting, editing, research), and how the AI contributions were integrated into their work. Additionally, they should declare any direct output from the AI that was included in the assignment and the extent to which the AI influenced their final submission. This documentation should also include reflections on ethical considerations and compliance with institutional guidelines, ensuring transparency and accountability in their academic practices.

09:00:59

group01@tdt4242.ntnu.no:

At what level of detail should this documentation exist to be useful without becoming overly burdensome?

09:01:02

iRE agent:

The documentation should strike a balance by requiring clear but concise entries. For each assignment, students should provide a brief summary (1-2 sentences) of the AI tools used, their specific contributions, and any direct outputs incorporated. This level of detail should capture essential information without overwhelming students, allowing them to reflect on their AI usage while maintaining focus on their academic work. Consistency in documentation format will also help streamline the process.

09:01:05

group01@tdt4242.ntnu.no:

How is this type of documentation currently handled, if at all?

09:01:12

iRE agent:

Currently, this type of documentation is often not systematically handled. Many institutions lack formal requirements for students to document AI usage, leading to inconsistencies across courses. Some instructors may encourage informal reflections or discussions about AI assistance, but there is typically no standardized process for submission. This absence of structured documentation can result in confusion and varying expectations, highlighting the need for a comprehensive approach like the AI Guidebook to establish clear guidelines and promote accountability in AI usage among students.

09:01:15

group01@tdt4242.ntnu.no:

Which parts of AI usage documentation and compliance checking could realistically be automated today?

09:01:26

iRE agent:

Automation could realistically handle the logging of AI tool usage, tracking specific interactions (e.g., prompts and outputs) within assignments. Compliance checking against institutional guidelines could be automated by implementing rule-based systems that analyze submitted documentation for adherence to defined policies. Additionally, systems could automatically generate usage reports for students, summarizing their AI interactions and flagging any potential compliance issues. This automation would streamline the documentation process while ensuring transparency and accountability.

09:01:29

group01@tdt4242.ntnu.no:

What technical, institutional, or legal limitations must be considered when automating these processes?

09:01:34

iRE agent:

Technical limitations include the need for robust integration with existing educational platforms and ensuring data privacy in accordance with regulations like GDPR. Institutional limitations may arise from varying policies across departments, requiring customization of automation tools to align with different guidelines. Legally, there are concerns regarding intellectual property rights, particularly about the ownership of AI-generated content and compliance with academic integrity standards. Addressing these limitations is essential to implement effective and compliant automation processes.

09:01:38

group01@tdt4242.ntnu.no:

When during the academic workflow should automated compliance checks run, and why?

09:01:44

iRE agent:

Automated compliance checks should run at multiple key points: first, during the submission of assignments, to verify adherence to AI usage documentation requirements; second, when students log their AI interactions, to ensure real-time compliance with institutional policies; and finally, at the end of the semester, to generate comprehensive reports for review by academic staff. This multi-stage approach ensures ongoing accountability, reinforces ethical practices throughout the academic process, and provides timely feedback to students on their compliance with guidelines.

09:01:47

group01@tdt4242.ntnu.no:

What actions should the system take if a potential policy violation is detected?

09:01:55

iRE agent:

If a potential policy violation is detected, the system should first issue an alert to the student, providing details of the concern and guidance on how to address it. Following this, the system should flag the submission for review by academic staff, enabling them to assess the situation and determine appropriate actions. Additionally, the system could offer educational resources on responsible AI use to help students understand the implications of their actions.

and prevent future violations. This approach fosters a learning environment while maintaining academic integrity.

09:01:58

group01@tdt4242.ntnu.no:

What kinds of visualizations would be useful for presenting AI usage data to students?

09:02:05

iRE agent:

Useful visualizations for presenting AI usage data to students could include:

1. **Usage Trends Over Time**: Line graphs showing the frequency and types of AI tools used throughout the semester, helping students identify patterns in their usage.
2. **Tool Contribution Breakdown**: Pie charts or bar graphs illustrating the proportion of different AI tools utilized for various tasks, allowing students to assess their reliance on specific tools.
3. **Assignment-Level Insights**: Visual summaries for each assignment that highlight the amount of AI assistance used, including direct outputs and contributions, fostering reflection on their academic practices.
4. **Compliance Status Dashboard**: A visual status indicator showing compliance with documentation requirements, helping students stay on track with institutional policies.

These visualizations can enhance understanding and encourage responsible AI usage among students.

09:02:09

group01@tdt4242.ntnu.no:

How should these visualizations support student reflection, learning, or behavior change?

09:02:13

iRE agent:

These visualizations should provide actionable insights that prompt students to reflect on their AI usage patterns and their alignment with academic integrity. By clearly presenting data, students can identify areas where they may rely too heavily on AI or fail to document their usage appropriately. This awareness can lead to informed discussions about responsible AI use and encourage adjustments in behavior, fostering a more balanced approach to integrating AI tools in their academic work. Additionally, incorporating reflective

prompts alongside the visualizations can further enhance their learning experience and guide behavior change.

09:02:16

group01@tdt4242.ntnu.no:

Should similar visual insights be available to instructors or administrators, and under what constraints?

09:02:21

iRE agent:

Yes, similar visual insights should be available to instructors and administrators to monitor compliance and support students effectively. However, access should be constrained to protect student privacy and data confidentiality.

Visualizations should aggregate data rather than display individual student details, ensuring that instructors can identify trends and potential issues without compromising personal information. Additionally, access should be limited to authorized personnel, with clear guidelines on how to use the insights to foster a supportive learning environment rather than punitive measures. This approach balances oversight with respect for student privacy.

09:02:24

group01@tdt4242.ntnu.no:

What ethical, legal, or institutional constraints must the system respect when handling student data and AI usage logs?

09:02:28

iRE agent:

The system must respect several key constraints:

1. **Data Privacy Regulations**: Compliance with laws such as GDPR or FERPA, which govern the collection, storage, and sharing of personal data, ensuring students' information is protected.
2. **Informed Consent**: Students should be informed about what data is collected, how it will be used, and should have the option to consent to this data collection.
3. **Confidentiality**: Maintaining the confidentiality of student identities in reports and visualizations, ensuring that individual data cannot be traced back to specific students.
4. **Institutional Policies**: Adhering to university-specific guidelines regarding data usage, intellectual property rights, and academic integrity standards.

5. **Ethical Considerations**: Ensuring that data handling practices promote fairness, transparency, and accountability, fostering an environment of trust between students and the institution.

These constraints are essential to maintain ethical standards and protect student rights while utilizing AI usage data.

09:02:35

group01@tdt4242.ntnu.no:

How should the system balance student privacy with the need for transparency and accountability?

09:02:38

iRE agent:

The system should implement data anonymization techniques to protect student identities while still providing aggregated insights for transparency and accountability. By focusing on trends and patterns rather than individual data points, the system can allow instructors and administrators to monitor compliance effectively without compromising privacy. Additionally, clear communication about data usage policies and the purpose of data collection can enhance trust, ensuring that students understand how their information contributes to a supportive learning environment. Establishing strict access controls and regular audits can further safeguard privacy while maintaining accountability.

09:02:40

group01@tdt4242.ntnu.no:

What level of explainability should the system provide when making compliance-related decisions?

09:02:45

iRE agent:

The system should provide a high level of explainability regarding compliance-related decisions. This includes clear, understandable rationales for alerts or flags raised concerning potential policy violations, detailing the specific criteria or rules that triggered the decision.

Students should receive feedback that outlines what aspects of their AI usage were deemed non-compliant and why, enabling them to learn from their mistakes and adjust their behavior. Additionally, the system should include accessible documentation of the compliance rules and decision-making processes, ensuring transparency and fostering a culture of accountability and continuous improvement among users.

09:02:48

group01@tdt4242.ntnu.no:

How would you define a successful AI Guidebook in terms of learning outcomes, system functionality, and responsible AI use?

09:02:51

iRE agent:

A successful AI Guidebook would achieve the following:

1. **Learning Outcomes**: Students demonstrate a clear understanding of responsible AI usage, reflected in their ability to articulate ethical considerations, document AI interactions accurately, and apply institutional guidelines effectively.
2. **System Functionality**: The system provides seamless logging of AI usage, real-time compliance checks, and intuitive visualizations that support self-reflection. It should facilitate easy access to ethical guidelines and educational resources, enhancing user engagement and understanding.
3. **Responsible AI Use**: The Guidebook promotes a balanced integration of AI tools in academic work, encouraging critical thinking and independent learning. Students would use AI responsibly, understanding its role as a support tool rather than a substitute for their own effort and creativity.

Overall, success is measured by improved academic integrity, enhanced student learning experiences, and positive feedback from both students and faculty.

09:02:56

group01@tdt4242.ntnu.no:

Are there measurable KPIs the system should track, and how should these be used by the institution?

09:02:59

iRE agent:

Yes, the system should track several key performance indicators (KPIs):

1. **Compliance Rates**: Percentage of students meeting AI documentation requirements for assignments, helping the institution assess adherence to academic integrity policies.
2. **Frequency of AI Tool Usage**: Monitoring how often and which AI tools are used, providing insights into student reliance on AI in their studies.

3. **Incident Reports**: Number of flagged compliance issues or potential violations, helping identify areas where additional education or support may be needed.

4. **User Engagement Metrics**: Tracking interaction with educational resources and the Guidebook itself, indicating the effectiveness of the system in promoting responsible AI use.

These KPIs should be used by the institution to evaluate the effectiveness of the AI Guidebook, inform policy adjustments, and enhance support for students. Regular analysis of these metrics can drive continuous improvement in both the system and overarching academic integrity initiatives.

09:03:04

group01@tdt4242.ntnu.no:

What risks or unintended consequences should be avoided when deploying such a system?

09:03:09

iRE agent:

Key risks and unintended consequences to avoid include:

1. **Over-reliance on AI Tools**: Students may become dependent on AI for completing assignments, undermining their critical thinking and problem-solving skills.

2. **Privacy Breaches**: Inadequate data protection measures could lead to unauthorized access to sensitive student information, violating privacy regulations and trust.

3. **Misinterpretation of Compliance Data**: Inaccurate or misleading compliance alerts could create unnecessary anxiety among students or penalize those who are actually adhering to guidelines.

4. **Inequitable Access**: Not all students may have the same level of access to AI tools or the Guidebook, potentially exacerbating disparities in academic performance.

5. **Resistance to Adoption**: If students perceive the system as punitive rather than supportive, it could hinder engagement and utilization of the Guidebook.

Mitigating these risks requires careful planning, ongoing stakeholder engagement, and responsive design based on user feedback.

09:03:14

30.01.26

group01@tdt4242.ntnu.no:

How do you (roughly) imagine the website to look?

09:24:28

iRE agent:

I cannot provide a visual concept for the website. However, I can help outline functional requirements that focus on user interface elements. The system shall include a personal dashboard for usage visualization, clear sections for ethical guidelines, AI usage logging, and assignment-level declarations. Additionally, usability should be prioritized to ensure students can navigate easily and access information without confusion. If you have specific design elements in mind, we can explore their functional implications.

09:24:31

group01@tdt4242.ntnu.no:

What pages should the website be divided into?

09:25:19

iRE agent:

The website should be divided into the following key sections:

1. ****Home Page****: An overview of the application and its purpose.
2. ****Dashboard****: A personal area for students to visualize their AI usage patterns.
3. ****AI Usage Logging****: A section for students to log their AI tool usage.
4. ****Assignment-Level Declarations****: A page dedicated to declaring AI assistance for specific assignments.
5. ****Ethical Guidelines****: Clear information on ethical use of AI tools and academic integrity.
6. ****Compliance Checks****: Automated feedback on adherence to institutional policies.
7. ****Help/Support****: Resources and FAQs for user assistance.

Each section should be designed with usability in mind to ensure easy navigation and accessibility of information.

09:25:24

group01@tdt4242.ntnu.no:

What should be on the "home page"?

09:27:44

iRE agent:

The home page should include:

1. ****Introduction****: A brief overview of the AI Guidebook's purpose and benefits for students.

2. ****Key Features****: Highlight main functionalities, such as the personal dashboard, usage logging, and ethical guidelines.
3. ****Navigation Menu****: Clear links to all major sections of the website for easy access.
4. ****Latest Updates****: Announcements or news related to AI tools and academic integrity.
5. ****Call to Action****: Encouragement for students to register or log in to start using the application.

This layout will ensure that visitors quickly understand the platform's value and how to navigate it effectively.

09:27:47

group01@tdt4242.ntnu.no:

How much load should the website be able to handle?

09:48:41

iRE agent:

The website should be designed to handle concurrent users based on expected student enrollment and usage patterns. A preliminary estimate might suggest supporting at least 500 concurrent users during peak hours, considering the scale of student engagement. Additionally, performance metrics should focus on response times under load, aiming for page load times of less than 2 seconds. It's essential to conduct load testing to refine these estimates and ensure robust performance under varying conditions.

09:48:46

group01@tdt4242.ntnu.no:

How much downtime is acceptable?

09:49:46

iRE agent:

The acceptable downtime for the website should ideally be minimal, targeting a maximum of 1-2 hours per month for maintenance and updates. This translates to a high availability goal of 99.9% uptime, ensuring that students have reliable access to the AI Guidebook at all times. Any extended downtime should be communicated in advance, with contingency plans in place to minimize impact on users.

09:49:49

These are the GitHub Copilot logs. These normally include all the code changes made while it works on the code base, but these are emitted here as they are quite big:

Task 2.2 AI Log

Participant: User

Assistant: GitHub Copilot

Request 1:

Participant

could you please follow the instructions given in the file "exercise2" and make me the application

Assistant

Used 2 references

- ≡ exercise2.instructions.md
- ≡ instructionsList

✓ Optimizing tool selection...

I'll build the full application described in your **exercise2** instructions end-to-end (backend + frontend + policy engine + GDPR delete flow), starting by checking the workspace and then scaffolding the project structure to match the required architecture.

I confirmed the workspace is empty, so I'll scaffold a clean full-stack project and then implement each requirement layer in dependency order so everything runs cohesively.

I'm now creating the full project skeleton (root, backend, frontend, and feature folders) and then I'll fill in backend logic first so the frontend can consume real APIs.

Scaffold files are in place. Next I'm implementing backend core (**models**, **services**, **controllers**, **routes**, and middleware) so the compliance and logging APIs are fully functional before wiring the UI.

The data models and validators are done. I'll now add logging/compliance services plus route/controller wiring, including batch compliance and GDPR delete behavior.

I'm adding the non-blocking auto-logging middleware and bootstrapping the Express app so requests can be logged asynchronously without delaying responses.

Backend implementation is now in place including async auto-logging and compliance evaluation. Next I'll build the React feature module (`usage-compliance`) with hooks, retrying API client, and educational feedback components.

I'm wiring the feature components and hooks now: form submission, debounced compliance checks, and educational-first feedback rendering with empty/error states.

The UI feature is complete. I'll add sample policy configuration plus setup documentation that maps each implementation piece to FR/NFR so you can run and extend it immediately.

I'm running a quick validation pass now (problem diagnostics first, then install/build checks) to catch any implementation errors before handoff.

Implemented the full application from your `exercise2` spec in a new monorepo with backend + frontend, including GDPR deletion, async auto-logging, configurable policy rules, compliance checking, and educational feedback UI.

What Changed

- Backend API with Mongo/Mongoose models, services, controllers, routes, and middleware is in place
- Frontend feature module implemented with form, debounced compliance checks, educational feedback, ethical guidance card, retrying API client, and error boundaries:
- Example policy JSON and setup/docs added:

Validation

- Ran `npm install` successfully at workspace root.
- Ran `npm run build --workspace frontend` successfully.
- VS Code diagnostics currently report no code errors.

Run

- Start backend: `npm run dev --workspace backend` (requires MongoDB + `backend/.env`).
- Start frontend: `npm run dev --workspace frontend`.

Want me to add a small seed script next to import directly into Mongo with one command?

Response time: 309.95 seconds

Task 2.3 AI Log

Participant: User

Assistant: GitHub Copilot

Request 1

Participant

can you perform a code review of the code base? this is what you should look after:

You will apply code review practices to identify and reason about code smells—structural weaknesses that can reduce maintainability, readability, reliability, or performance.

Examples of code smells include (but are not limited to): long or overly complex components, duplicated logic, primitive obsession, large classes, long parameter lists, temporary fields, shotgun surgery, dead or unreachable code, excessive comments, deeply nested JSX, mixing UI and business logic, and missing error handling.

for every code smell you identify please list them in this format:

- (1) Name of the smell
- (2) Short description
- (3) Identify method
- (4) Relevant original code snippets

if its easier please give me the list as its own document

Assistant

Used 1 references

≡ instructionsList

✓ Optimizing tool selection...

I'll review the repository for maintainability and reliability code smells, then produce a separate document with each finding in the exact format you requested, including code snippets and where I found them. I'll first load the project-specific instructions and scan both backend and frontend code for high-confidence issues.

I've loaded the project guidance and will now run a structured scan of backend and frontend files to identify concrete code smells with supporting snippets. Next I'll compile the findings into a standalone review document in the repo.

I'm collecting the core source files now so the findings are grounded in actual code, not assumptions. I'll focus on services, controllers, middleware, hooks, and UI components where smells most often appear.

I've finished the backend pass and found several likely smells already; next I'll inspect models/routes/frontend to confirm impact and gather precise snippets for the report.

I've completed both backend and frontend inspection and identified a set of high-confidence smells with concrete locations. I'm now drafting the findings in a dedicated markdown document using your requested 4-part format per smell.

I'm creating the review file now at the repo root so you can submit or share it directly. It will contain each smell in your exact (1)-(4) format with code excerpts.

Created a standalone review document with 10 code smells in your exact requested format:

- (1) Name of the smell
- (2) Short description
- (3) Identify method
- (4) Relevant original code snippets

If you want, I can now produce a prioritized fix plan (high/medium/low impact) for these findings and implement the top 2-3 refactors directly.

Response time: 145.54 seconds

Request 2

Participant

ok we performed a manual code review as well. can you use the instruction file "code_review" to find the identified code smells in the codebase and fix them. the ones we found manually have the code snippets as images so you won't be able to see them, but hopefully you can find them anyway.

Assistant

Used 2 references

≡ code_review.instructions.md
≡ instructionsList

I'll implement fixes for the manually identified smells from the `code_review` instruction file first, then handle any closely related duplication issues that are safe to fix in the same pass. I'm starting by re-reading the instruction and the impacted files to map each smell to concrete code edits.

I've mapped the manual findings to concrete fixes and I'm proceeding with a focused refactor in four parts:

1. Extract policy-rule querying into a repository module and reuse it from both `policyController` and `complianceService` (fix low cohesion + duplicated query logic).
2. Centralize usage-payload validation in `logController` so both endpoints call one helper (fix duplicated validation).
3. Reuse
in
instead of manually reading header-only identity (fix duplication/inconsistency).
4. Keep Express error middleware signature but mark unused args explicitly (`_req`, `_next`) to eliminate the dead/unused-args smell cleanly.

After edits, I'll run a quick error check on touched backend files and report exactly what changed.

I'm now applying the refactor edits across backend files in one pass so the new repository abstraction and controller cleanup stay consistent. After this patch, I'll run diagnostics on the modified files.

Core fixes are in place: policy querying is now centralized in a repository, log payload validation is deduplicated, delete-user resolution reuses the shared helper, and unused args are explicitly marked. I'll run an error check on changed backend files now to ensure everything compiles cleanly.

Implemented fixes for the manually identified code smells and verified there are no diagnostics in changed files.

Validation

- Checked errors for all touched files with `get_errors`; no issues found.

Want me to continue with the next accepted smells from your review list (e.g., per-item batch validation or sequential async in batch compliance)?

Response time: 94.25 seconds

Request 3

Participant

yes please continue

Assistant

Used 2 references

≡ code_review.instructions.md
≡ instructionsList

I'll continue by fixing the next high-impact accepted smells in backend first: per-item batch validation, broad catch diagnostics, and sequential batch processing. Then I'll run diagnostics again and summarize what remains.

I've confirmed the current code and will now patch three backend areas: strict per-item validation for batch checks, parallelized batch compliance evaluation, and better diagnostics when rule evaluation fails.

I'm applying a second pass to reduce the remaining accepted smells on the frontend/backend boundary: remove payload-mapping duplication in, centralize required field lists in shared types, and actually use rule-level educational messages in compliance responses.

I've applied the second wave of fixes across backend + frontend. Next I'm running diagnostics on the touched files to ensure no regressions before I summarize remaining smells.

Progress

- Continued and fixed additional accepted smells, with clean diagnostics on all touched files.

Already fixed in prior pass

- Cohesion/query duplication via repository extraction in
and reuse from
and
.
- Unused args cleanup in
.

Want me to finish the last remaining smells as well (for example, unused
and further simplification of
)?

Response time: 111.34 seconds

Request 4

Participant

yes please finish

Assistant

Used 2 references

≡ code_review.instructions.md
≡ instructionsList

I'll finish the remaining practical smells now, starting with the confirmed unused code path () and any now-unused API/helpers it leaves behind. Then I'll run a final diagnostics sweep and give you a complete "done" status.

I confirmed

,
, and
are currently unreferenced in the frontend code path, so I'll remove that dead client code now to close out the unused-path smell cleanly.

I've removed the remaining unused frontend path and related dead API helpers. I'll now run one final diagnostics + usage scan to confirm there are no leftover references or new issues.

Done

- Finished the remaining practical fixes from your review list and verified no new diagnostics.
- Removed unused frontend code path (**refreshLogs**) and related unused API helpers in
and

If you want, I can do one final polish pass specifically for the "long component" smell by splitting
into smaller presentational subcomponents.

Response time: 78.39 seconds

Request 5

Participant

if its possible yes

Assistant

Used 2 references

≡ code_review.instructions.md
≡ instructionsList

I'll do a lightweight refactor of the form component by extracting its repeated JSX sections into small presentational subcomponents, without changing behavior or API. After that I'll run diagnostics to confirm everything is clean.

I'm reading the latest form file now so I can apply a single focused patch that only restructures JSX sections.

I'm now applying the refactor to split the JSX into small field-section components while keeping the submit/validation logic intact.

Completed. I refactored the long form component into smaller presentational sections while preserving behavior and props.

If you want, I can also update CODE_REVIEW_SMELLS.md with a "fixed status" section per smell so your submission clearly shows what was addressed.

Response time: 69.60 seconds

Task 3.3 AI Log

Participant: User

Assistant: GitHub Copilot

Request 1

Participant

In this task, you will move from test design to test implementation. Using the existing codebase, you will implement executable unit tests. The objective is to achieve high structural test coverage, specifically: Line coverage: $\geq 60\%$ Branch coverage: $\geq 70\%$ Function/method coverage: $\geq 80\%$ You must write or generate real, runnable unit tests that target critical functions and logic in the system. The test suite must execute locally without errors, produce reproducible coverage results, and demonstrate meaningful testing of edge cases and branch conditions.

Outcome

1. Executable test files committed to your GitHub repository. Tests stored under /tests/ or /src/tests/ Tests must include assertions, setup/teardown where relevant, mocks or stubs, edge-case coverage, and explicit branch testing
2. Coverage report, generated by running: `npm run test -- --coverage`

Evaluation Criteria

Achievement of coverage targets: Test suites meet or exceed the specified line, branch, and function coverage thresholds
Quality of unit tests: Tests are deterministic, maintainable, clearly structured, and focused on behaviour rather than implementation details

use the instruction file "test_plan", to do this task. This is the description of the test plan just so you get an understanding of it, which was the previous task:

Description

In this task, you will design a structured test suite for the AIGuidebook system that reflects professional quality-assurance (QA) practices. Your test set must include a balanced combination of unit tests, integration tests, system tests, and non-functional requirement (NFR) tests. You will also define test coverage criteria for the system, including coverage of lines of code (LOC), branches/paths, and requirements. In addition, you must create a traceability matrix that links each requirement group selected from Task 2.1 to one or more test cases and, where possible, to specific implementation components.

Outcome

A test plan document including: Test Case Table (cover all the given requirement group) using the following structure: | Test ID | Test Type | Input / Trigger | Expected Output | Test Method | Pass/Fail Criteria | Notes | Coverage Definition briefly specifying coverage goals, for example: LOC coverage (e.g. $\geq 65\%$) Branch/path coverage (e.g. each conditional evaluated true and false) Requirement coverage (100%) NFR coverage (at least one test per NFR category, such as security or usability) Traceability Matrix linking: Requirement $\rightarrow$ Test Case(s) $\rightarrow$ Implementation reference (file or component, if applicable)

Assistant

Used 2 references

≡ test_plan.instructions.md
≡ instructionsList

✓ Optimizing tool selection...

I'm going to inspect the existing test/tooling setup and key backend logic first, then I'll implement runnable unit tests aligned with your `test_plan` requirements and verify coverage with `npm run test -- --coverage`.

I've loaded the required instruction files and next I'm mapping the current test setup plus the highest-value backend units (`services`, `middleware`, `utils`, models) so we can hit your coverage targets with deterministic tests.

The automated code search couldn't resolve paths, so I'm switching to direct workspace inspection and will read the backend files manually before writing tests.

I confirmed there is currently no test framework configured, so next I'm reading the backend logic and then I'll add a unit-test setup plus tests for branch-heavy functions.

I found that `autoLoggingMiddleware.js` does not exist in this codebase, so I'll focus unit coverage on the actual implemented modules and branches that are present. I'm now reading the remaining backend files to build a complete, accurate test suite.

I now have enough context and the next step is implementation plus verification.

Plan:

1. Add a backend test runner setup (`vitest`) and wire root `npm run test` so your required command works.
2. Implement unit tests under `backend/src/tests/` for branch-heavy logic: `complianceService`, `loggingService`, `validators`, `policyRuleRepository`, `logController`, and `policyController`.
3. Run `npm run test -- --coverage`, inspect line/branch/function metrics, and add any extra edge-case tests needed to meet thresholds.
4. Summarize exactly what was added and report the reproducible coverage results.
I'm starting file edits now: first wiring a test command and coverage config, then I'll add focused unit test files for each critical backend module.

I've started writing the unit suite and next I'm adding tests that heavily exercise retry and query-branch logic in `loggingService`, then controller behaviors with mocked dependencies.

I've added a full backend unit-test suite. Next I'm installing test dependencies and running the required coverage command so we can confirm whether all thresholds are met and patch any gaps.

Ran tool

Completed with input: `npm run test -- --coverage`

The first test run exposed a Vitest ESM mocking issue (`vi.mock` hoisting). I'm now patching the mocks to use `vi.hoisted(...)` so suites load correctly, and I'll also fix root script argument forwarding for `--coverage`.

I've fixed the failing test branches and made coverage generation explicit in the backend script; I'm re-running the full command now to confirm final metrics.

Implemented a runnable backend unit test suite aligned to your `test_plan`, and verified coverage with the exact command you requested.

Coverage run

Command executed:

```
npm run test -- --coverage
```

Result:

- Test files: 8 passed
- Tests: 53 passed
- Coverage:
 - Line/statement: 70.41%
 - Branch: 86.27%
 - Function: 84.61%

This meets your targets:

- Line $\geq$ 60% ✓
- Branch $\geq$ 70% ✓
- Function/method $\geq$ 80% ✓

How this maps to your test-plan intent

- Branch-heavy compliance logic covered:
 - Unknown rule type path
 - Malformed rule fallback handling
 - Submission aggregate warning/ok paths
- Controller edge/error paths covered:
 - Validation errors, auth/user resolution, forbidden delete, malformed batch logs, success responses, service failures
- Service robustness covered:
 - Retry with backoff and exhausted retries
 - Query building and limit bounding
 - Error handler production/non-production behavior
 - DB config validation paths

Notes

- `npm run test -- --coverage` shows npm warnings about unknown `--coverage` config forwarding, but tests and coverage execute correctly because backend test script runs `vitest run --coverage`.
- The current codebase does not contain `backend/src/middleware/autoLoggingMiddleware.js`, so tests were implemented against the actual existing modules.

If you want, I can also make a single commit with a clean message for this test implementation.

Response time: 538.13 seconds

Request 2

Participant

if you look at the task/instructions mentioned in my previous message, the frontend should also be covered by tests please add these

Assistant

Used 2 references

≡ test_plan.instructions.md

≡ instructionsList

I'll extend the suite to cover frontend behavior as well: first I'll inspect current frontend setup and components/hooks, then add Vitest + RTL tests and verify they run with coverage.

I've mapped the frontend logic and I'm now wiring a proper frontend test environment (`jsdom` + React Testing Library) and adding unit tests for components, hooks, services, and utils so coverage is meaningful, not just smoke tests.

I've updated scripts/config so root testing now includes frontend too. Next I'm adding frontend test files (setup, utils, service, hooks, and components) with explicit branch and edge-case assertions.

Frontend test files are in place. I'm installing the new frontend test dependencies and then I'll run the full root coverage command to verify both backend and frontend pass.

Ran tool

Completed with input: `npm run test -- --coverage`

I found the blocker: frontend Vitest was running in Node mode (no `environment`), so I'm adding an explicit `frontend/vitest.config.js` for `jsdom` and then tightening one async rejection test that caused an unhandled promise warning.

Ran tool

Completed with input: `npm run test --workspace frontend -- --coverage`

I'm now fixing the remaining frontend failures: adding explicit imports in JSX test files (required by this transform setup) and correcting two hook test expectations for async state timing.

I fixed the remaining failures by adding test cleanup and more reliable assertions; I'm running frontend coverage once more to confirm all tests are green before re-running the root command.

Implemented frontend test coverage end-to-end and verified it runs with your required command.

Verification

I ran:

```
npm run test -- --coverage
```

Result:

- Backend: 53/53 tests passed
- Frontend: 26/26 tests passed

Frontend coverage:

- Lines: 98.11%
- Branches: 92.07%
- Functions: 96.87%

So frontend is now covered with executable tests and meaningful branch/edge-case assertions, aligned with the test-plan intent.

If you want, I can now create a single commit with a clean message for all backend+frontend test additions.

Response time: 491.46 seconds

Request 3

Participant

im seeing now that the coverage report needs to be in HTML, please change/add this

Assistant

Used 2 references

≡ test_plan.instructions.md
≡ instructionsList

I'll make HTML coverage generation explicit in the test scripts (not just config-driven) and then run the command once to confirm the HTML reports are produced reliably.

Updated so HTML coverage is explicitly generated by the test scripts, and confirmed it works.

Verified

I ran:

```
npm run test -- --coverage
```

Both backend and frontend passed, and HTML reports were generated

If you want, I can also add a root helper script like `test:coverage:open` to quickly open both HTML reports.

Response time: 53.72 seconds